TP préparatoire — Recherche vectorielle

TD1 §5 · S2.02 · UniCA 2025-2026

Erwan Nalin · Avril 2026

Préambule

Ce document regroupe les réponses aux exercices **12**, **13** et **14** de la section *5 — Application : recherche d'information* du TD1 *Espaces euclidiens* (R107).

Les implémentations Python associées sont dans :

- `librairie.py` — fonctions vectorielles + lemmatisation/filtrage,
- `moteur_de_recherche.py` — moteur de base (modifié pour gérer la requête « pomme de terre », Ex 14.2),
- `moteur_de_recherche_ameliore.py` — moteur avec pondération TF-IDF.

Exercice 12 — Questions théoriques

1. À quoi correspond n concrètement ?

n est la **taille du vocabulaire** : c'est le nombre de mots **distincts** présents dans au moins un document du corpus indexé. Chaque dimension de l'espace $\mathbb{R}^n$ correspond à un mot du vocabulaire ; la i-ième coordonnée d'un vecteur indique le nombre d'occurrences du i-ième mot dans le document (ou la requête).

Sur le corpus de 26 recettes du TP, le programme reporte `661 mots trouvés` : on travaille donc dans $\mathbb{R}^{661}$.

2. Pourquoi le cosinus et non le produit scalaire ?

Le produit scalaire $\vec{r} \cdot \vec{d}$ est **sensible à la norme** des vecteurs : un long document riche en mots variés obtient mécaniquement un score élevé, même si la requête n'y est pas particulièrement représentée.

Le cosinus, lui, normalise par les longueurs :

$$\cos(\theta) = \frac{\vec{r} \cdot \vec{d}}{\|\vec{r}\| \cdot \|\vec{d}\|}$$

Il ne dépend que de la **direction** des vecteurs, c'est-à-dire du *profil relatif* des mots, pas de leur abondance absolue. Une recette courte et très ciblée sur la requête peut donc battre une recette longue et générique — c'est exactement ce qu'on veut.

3. Cosinus de 0 ? Cosinus de 1 ?

Valeur	Interprétation géométrique	Interprétation sémantique
$\cos(\theta) = 0$	les vecteurs sont **orthogonaux**	le document et la requête n'ont **aucun mot en commun**
$\cos(\theta) = 1$	les vecteurs sont **colinéaires** (même direction)	le document a **exactement le même profil de mots** que la requête (à un facteur d'échelle près)

Entre les deux, plus le cosinus est proche de 1, plus le document est pertinent.

4. Pourquoi normaliser revient au même qu'utiliser le cosinus ?

Soient $\vec{u} = \frac{\vec{r}}{\|\vec{r}\|}$ et $\vec{v} = \frac{\vec{d}}{\|\vec{d}\|}$ les vecteurs normalisés. Alors :

$$\vec{u} \cdot \vec{v} = \frac{\vec{r}}{\|\vec{r}\|} \cdot \frac{\vec{d}}{\|\vec{d}\|} = \frac{\vec{r} \cdot \vec{d}}{\|\vec{r}\| \cdot \|\vec{d}\|} = \cos(\theta)$$

Calculer le **produit scalaire de vecteurs normalisés** est strictement équivalent à calculer le **cosinus des vecteurs originaux**. C'est la stratégie adoptée par `moteur_de_recherche.py` : on normalise une bonne fois pour toutes les vecteurs des documents au démarrage, puis chaque requête se contente d'un produit scalaire — beaucoup plus rapide à grande échelle.

Exercice 13 — Calculs manuels

Données

Mots	d_1	d_2	d_3	d_4	d_5
aubergine	0	3	4	7	3
courgette	2	1	3	8	5
poivron	2	3	2	0	1
tomate	12	6	5	17	6

L'ordre des coordonnées est **(aubergine, courgette, poivron, tomate)**.

Question 1.a — Vecteurs normalisés

On calcule d'abord la norme euclidienne $\|\vec{d}_i\| = \sqrt{\sum_j d_{i,j}^2}$ de chaque document :

$$\begin{aligned}
\|\vec{d}_1\|^2 &= 0^2 + 2^2 + 2^2 + 12^2 = 152 & \|\vec{d}_1\| &= \sqrt{152} \approx 12{,}329 \\
\|\vec{d}_2\|^2 &= 9 + 1 + 9 + 36 = 55 & \|\vec{d}_2\| &= \sqrt{55} \approx 7{,}416 \\
\|\vec{d}_3\|^2 &= 16 + 9 + 4 + 25 = 54 & \|\vec{d}_3\| &= \sqrt{54} \approx 7{,}348 \\
\|\vec{d}_4\|^2 &= 49 + 64 + 0 + 289 = 402 & \|\vec{d}_4\| &= \sqrt{402} \approx 20{,}050 \\
\|\vec{d}_5\|^2 &= 9 + 25 + 1 + 36 = 71 & \|\vec{d}_5\| &= \sqrt{71} \approx 8{,}426
\end{aligned}$$

Les vecteurs normalisés $\vec{u}_i = \vec{d}_i / \|\vec{d}_i\|$ sont :

	$u_{i,1}$ (aubergine)	$u_{i,2}$ (courgette)	$u_{i,3}$ (poivron)	$u_{i,4}$ (tomate)
$\vec{u}_1$	0,0000	0,1622	0,1622	0,9733
$\vec{u}_2$	0,4045	0,1348	0,4045	0,8090
$\vec{u}_3$	0,5443	0,4082	0,2722	0,6804
$\vec{u}_4$	0,3492	0,3990	0,0000	0,8479
$\vec{u}_5$	0,3560	0,5934	0,1187	0,7121

Question 1.b — Requête « tomate poivron »

Le vecteur de requête vaut $\vec{r} = (0, 0, 1, 1)$, de norme $\|\vec{r}\| = \sqrt{2}$, donc :

$$\vec{r}_{\text{norm}} = \left(0,\ 0,\ \tfrac{1}{\sqrt{2}},\ \tfrac{1}{\sqrt{2}}\right) \approx (0;\ 0;\ 0{,}7071;\ 0{,}7071)$$

La similarité cosinus avec chaque document est $\mathrm{sim}(\vec{r}, \vec{d}_i) = \vec{r}_{\text{norm}} \cdot \vec{u}_i = \frac{r_3 d_{i,3} + r_4 d_{i,4}}{\sqrt{2} \cdot \|\vec{d}_i\|}$:

$$\begin{aligned}
\mathrm{sim}(\vec{r}, \vec{d}_1) &= \frac{0+2+12}{\sqrt{2}\sqrt{152}} = \frac{14}{\sqrt{304}} \approx 0{,}8029 \\
\mathrm{sim}(\vec{r}, \vec{d}_2) &= \frac{3+6}{\sqrt{2}\sqrt{55}} = \frac{9}{\sqrt{110}} \approx 0{,}8581 \\
\mathrm{sim}(\vec{r}, \vec{d}_3) &= \frac{2+5}{\sqrt{2}\sqrt{54}} = \frac{7}{\sqrt{108}} \approx 0{,}6736 \\
\mathrm{sim}(\vec{r}, \vec{d}_4) &= \frac{0+17}{\sqrt{2}\sqrt{402}} = \frac{17}{\sqrt{804}} \approx 0{,}5996 \\
\mathrm{sim}(\vec{r}, \vec{d}_5) &= \frac{1+6}{\sqrt{2}\sqrt{71}} = \frac{7}{\sqrt{142}} \approx 0{,}5874
\end{aligned}$$

Classement (sac de mots brut) : $\boxed{d_2 \succ d_1 \succ d_3 \succ d_4 \succ d_5}$

Question 2.a — Coefficients de pertinence (IDF)

Le coefficient de pertinence du mot i est :

$$c_i = \log\left(\frac{D}{d_i}\right)$$

avec $D = 5$ documents et d_i = nombre de documents contenant le mot i.

Mot	Présent dans	d_i	$c_i = \log(5/d_i)$
aubergine	d_2, d_3, d_4, d_5	4	$\log(5/4) \approx 0{,}2231$
courgette	tous	5	$\log(1) = 0$
poivron	d_1, d_2, d_3, d_5	4	$\log(5/4) \approx 0{,}2231$
tomate	tous	5	$\log(1) = 0$

> **Lecture :** *courgette* et *tomate* sont présents dans tous les documents → ils ne discriminent rien → leur coefficient est nul. Seuls *aubergine* et *poivron* portent de l'information.

Question 2.b — Nouveau classement avec TF-IDF

On applique les coefficients : $\vec{d}_i^{\,\text{tfidf}} = (a_i \cdot 0{,}2231,\ 0,\ p_i \cdot 0{,}2231,\ 0)$ où a_i et p_i sont les comptages d'aubergine et poivron.

Document	$\vec{d_i}^{\text{ tfidf}}$	$\|\vec{d_i}^{\text{ tfidf}}\|$
d_1	$(0;\ 0;\ 0{,}4463;\ 0)$	$0{,}4463$
d_2	$(0{,}6694;\ 0;\ 0{,}6694;\ 0)$	$0{,}9466$
d_3	$(0{,}8926;\ 0;\ 0{,}4463;\ 0)$	$0{,}9981$
d_4	$(1{,}5620;\ 0;\ 0;\ 0)$	$1{,}5620$
d_5	$(0{,}6694;\ 0;\ 0{,}2231;\ 0)$	$0{,}7055$

Le vecteur de requête pondéré devient $\vec{r}^{\text{ tfidf}} = (0,\ 0,\ 0{,}2231,\ 0)$, de norme $0{,}2231$, donc $\vec{r}^{\text{ tfidf}}_{\text{norm}} = (0, 0, 1, 0)$: **seule la composante poivron compte**.

Les similarités sont alors la 3e coordonnée des $\vec{u_i}^{\text{ tfidf}}$:

$$\begin{aligned}
\text{sim}^{\text{ tfidf}}(\vec{r}, \vec{d_1}) &= \tfrac{0{,}4463}{0{,}4463} = 1{,}0000 \\
\text{sim}^{\text{ tfidf}}(\vec{r}, \vec{d_2}) &= \tfrac{0{,}6694}{0{,}9466} \approx 0{,}7071 \\
\text{sim}^{\text{ tfidf}}(\vec{r}, \vec{d_3}) &= \tfrac{0{,}4463}{0{,}9981} \approx 0{,}4472 \\
\text{sim}^{\text{ tfidf}}(\vec{r}, \vec{d_4}) &= 0 \\
\text{sim}^{\text{ tfidf}}(\vec{r}, \vec{d_5}) &= \tfrac{0{,}2231}{0{,}7055} \approx 0{,}3162
\end{aligned}$$

Classement TF-IDF : $\boxed{d_1 \succ d_2 \succ d_3 \succ d_5 \succ d_4}$

Question 2.c — Différences avec le classement précédent

Document	Sac de mots	TF-IDF	Variation
d_1	2e	**1er**	$\nearrow$
d_2	**1er**	2e	$\searrow$
d_3	3e	3e	$=$
d_4	4e	**5e**	$\searrow\searrow$
d_5	5e	4e	$\nearrow$

Explications :

- **d_1 monte au 1er rang.** Sa pertinence dans le modèle brut était dominée par les 12 occurrences de *tomate*, mais comme *tomate* a un poids nul, c'est désormais sa proportion de *poivron* (élevée par rapport au reste de son contenu, et *aubergine* absente) qui le porte. Tous ses mots non-nuls — *poivron* — coïncident avec un mot discriminant de la requête.
- **d_4 s'effondre au dernier rang.** Il contient principalement *aubergine* (7) et *tomate* (17) mais **0 poivron**. Le poids de *tomate* étant nul, et la requête ne contenant pas *aubergine*, sa similarité

tombe à 0.

- d_2 **recule.** Il était premier grâce à un bon mélange de *poivron* (3) et *tomate* (6), mais TF-IDF lui retire toute contribution de *tomate*. Sa pertinence se réduit à *poivron* concurrencé par *aubergine*, ce qui le fait passer derrière d_1 qui n'a *que* du poivron.

> **Morale :** la pondération TF-IDF a déplacé la mesure de pertinence depuis « combien de mots de la requête le document contient » vers « combien de **mots discriminants** ». C'est exactement l'effet recherché.

Exercice 14 — Implémentation Python

14.1 — Recherche de « tomate » et « tomate fromage »

L'exécution du moteur fourni `moteur_de_recherche.py` produit :

```
$ echo "tomate" | python3 moteur_de_recherche.py
- tartelettes_tomates_chevres.txt -> pertinence : 0.10101525445522107
- tarte_tomates.txt               -> pertinence : 0.09853292781642932
- crevettes_patates_douces.txt    -> pertinence : 0.0944911182523068
```

Les résultats sont cohérents : les recettes contenant explicitement la tomate remontent en tête. Pour `tomate fromage`, on récupère 5 résultats dont une recette de cheesecake (mot *fromage* présent) et plusieurs recettes mêlant tomate et fromage.

14.2 — Problème de « pomme de terre » et solution

Diagnostic

Avec le code initial, le mot **« de »** est inclus dans le vecteur de la requête comme dans celui de chaque document. Or *de* apparaît dans **toutes** les recettes (français usuel) : la requête `pomme de terre` est donc partiellement satisfaite par n'importe quelle recette française, ce qui pollue le classement et, dans certains corpus, fait remonter des documents non-pertinents.

Correction

J'ai ajouté à `librairie.py` une **liste de mots vides** (stopwords) et un filtrage à deux endroits :

1. **À l'indexation** (`lister_mots`) : les mots vides sont retirés des documents.
2. **À la construction de la requête** (`vecteur_requete`) : les mots vides sont retirés de la saisie utilisateur.

```
MOTS_VIDES = {
    'a', 'à', 'au', 'aux', 'avec', 'c', 'ce', 'ces', 'cet', 'cette',
    'd', 'dans', 'de', 'des', 'du', ...
}

def lister_mots(fichier):
    ...
    return [m for m in txt.split() if m not in MOTS_VIDES]

def vecteur_requete(requete, mots):
    l_mots = [m for m in requete.casefold().split() if m not in MOTS_VIDES]
    return [l_mots.count(mot) for mot in mots]
```

Validation

```
$ echo "pomme de terre" | python3 moteur_de_recherche.py
- gratin_pommes_terre.txt -> pertinence : 0.2716072381275556
- soupe.txt               -> pertinence : 0.0909090909090909
```

Le bon document remonte en tête avec un score net (~0,27 contre ~0,09 pour le concurrent), et la longue traîne de faux-positifs déclenchée par *de* a disparu.

> **Bonus.** Le moteur TF-IDF (`moteur_de_recherche_ameliore.py`) bénéficie également du filtrage et donne sur la même requête :
>
> ```
> - gratin_pommes_terre.txt -> pertinence : 0.3382085676667318
> - soupe.txt               -> pertinence : 0.1423885075031006
> ```
>
> Le score du gratin grimpe encore de 25 % grâce à la pondération IDF qui valorise *pomme* et *terre* (rares dans le corpus de recettes) face à des termes communs.

14.3 — Recherche de « pomme » : pistes d'amélioration

Constat

```
$ echo "pomme" | python3 moteur_de_recherche.py
Résultats de la recherche:
(aucun résultat)
```

Aucun résultat. Pourtant le corpus contient `gateau_pommes.txt`, `gratin_pommes_terre.txt`, et la recette de tarte mentionne *pommes* à plusieurs reprises.

Origine du problème

Le moteur travaille en *mots de surface* : il considère que **« pomme » et « pommes » sont deux mots distincts**. La requête au singulier ne matche aucun document, qui contiennent tous le pluriel.

Pistes d'amélioration

1. **Lemmatisation** *(approche retenue dans la SAÉ Java, version 0.3)*. On remplace chaque mot — dans les documents comme dans la requête — par sa forme canonique (lemme) :

   - *pomme, pommes, pomme* → **pomme**
   - *cuit, cuite, cuites, cuisant* → **cuire**

   Avantage : robuste, déterministe, peu coûteux en exécution (consultation O(1) dans une `HashMap`). Inconvénient : nécessite un dictionnaire de lemmatisation maintenu (un dictionnaire français tel que `french_dictionary.txt`, ~336 000 entrées, est fourni dans la SAÉ).

2. **Stemming** *(rabotage des suffixes)*. Plus brutal que la lemmatisation : on coupe juste les terminaisons connues (`-s`, `-es`, `-ent`, `-ait`…). L'algorithme de Porter ou son adaptation française (Snowball) ne demande aucun dictionnaire. Avantage : autonome. Inconvénient : crée parfois des collisions sémantiques (*cuisinier* et *cuisine* ramenés à la même racine).

3. **Recherche par préfixe / sous-chaîne**. Considérer qu'un mot m matche un mot w du document si w commence par m (ou contient m). Solution simple à implémenter, mais qui crée beaucoup de bruit (*pomme* matcherait *pommade*).

4. **Distance d'édition** (Levenshtein, Jaccard). Plus utile pour gérer les fautes de frappe. Pour les variations morphologiques, c'est moins efficace : *pomme* et *pommes* ont une distance de 1, mais *pomme* et *pommade* aussi.

5. **Synonymes / extension de requête**. Remplacer la requête par un cluster de mots équivalents (*pomme, pommes, pommier*…). Très efficace mais demande un thésaurus.

> **Conclusion.** Pour le projet HeRVé, la **lemmatisation par dictionnaire** est l'approche choisie : elle traite à la fois les pluriels, les conjugaisons, et les accords. C'est ce qu'implémente la version 0.3 du moteur Java.

Récapitulatif des modifications de code

Fichier	Action	Raison
`librairie.py`	Ajout de `MOTS_VIDES` (set de stopwords français)	Filtrer les mots vides pour Ex 14.2
`librairie.py`	Filtrage dans `lister_mots()`	Index propre, sans bruit
`librairie.py`	Filtrage dans `vecteur_requete()`	Requête robuste à `pomme de terre`
`moteur_de_recherche.py`	Inchangé (bénéficie du filtrage côté librairie)	API stable
`moteur_de_recherche_ameliore.py`	Inchangé (idem)	API stable

Tous les tests décrits ci-dessus ont été exécutés avec succès. Les formes de surface (singulier/pluriel) restent un problème ouvert qui sera résolu en V0.3 du projet Java par la lemmatisation.